

0812 Answer | 位模式的解释权

原题 29 题 · 上一节点 · 能力账本

从 0811 的“关键字值与来源段号要分开”，走到这组的“同一组位先问类型、位宽和存储顺序”。每题只增加裁决该题所需的动作。讲解已写不代表你已在限时遮答案复做中通关。

29 题考场入口

题	第一笔	结论
01	-9 先补到 32 位，再与 127 相加	D
02	分别把阶码、尾数还原成数值，再规格化和	C
03	四个等号分别问整数精度、截断、扩展、数量级	B
04	$-8.25 = -1.00001_2 \times 2^3$ ，填符号阶码尾数	C
05	unsigned short 的 FFFA 零扩展	B
06	最大有限单精度：阶码 254，尾数全 1	D
07	273 小端首字节 11；c 对齐到偶地址	D
08	C6400000 的符号位为 1，阶码 140	见题面核对
09	两个符号位都为 1，再比绝对值的阶码	A
10	最小补码先放负号，其余两个 1 放低位	B
11	区分对阶、右规、左规和最终溢出	D
12	8040 起小端第六个字节	A
13	FFFFFDF = -33，减 65	C
14	“规格化”限定指数域至少 1	A
15	前三指令字节不动，立即数写小端	A
16	同宽 FFFF 改按 signed short 解释	A
17	基址形成首址，再按大端找 LSB	B（位移解释见注）
18	同一位图分别按 int、float 解码	A
19	先对齐成员地址，再找小端字节	D
20	约分后看分母是否仅含因子 2	A
21	最高位负权，写补码两端	B
22	$0.4375 = 1.11_2 \times 2^{-2}$	B
23	$2^{16} - 8190$	A
24	指数零，按非规格化隐藏 0	A
25	短整数越界保留低 16 位后扩宽	B
26	范围与整数精度分别检查	C
27	负 short 转 32 位 unsigned 取模	D
28	指数 142、尾数 1.375	D
29	数组步长 20 + 小端字节偏移 1	C

01 | 2009-12：混合位宽，先算数值再写 32 位

12. 一个 C 语言程序在一台 32 位机器上运行。程序中定义了三个变量 x 、 y 和 z ，其中 x 和 z 为 `int` 型， y 为 `short` 型。当 $x = 127$ ， $y = -9$ 时，执行赋值语句 $z = x + y$ 后， x 、 y 和 z 的值分别是_____。

- A. $x = 0000007FH$ ， $y = FFF9H$ ， $z = 00000076H$
- B. $x = 0000007FH$ ， $y = FFF9H$ ， $z = FFFF0076H$
- C. $x = 0000007FH$ ， $y = FFF7H$ ， $z = FFFF0076H$
- D. $x = 0000007FH$ ， $y = FFF7H$ ， $z = 00000076H$

$x=127$ 、 $y=-9$ 是数值条件；`int`=32、`short`=16 决定赋值与运算时的位宽。草稿先写 $127+(-9)=118=0\times 76$ ，结果变量 z 为 32 位，机器数 `00000076H`，选 D。这一步比把两段十六进制串直接拼起来省事。

负的 `short` 如何参与，及为什么 `FFF7` 不会留在结果里

-9 的 16 位补码为 `FFF7H`。参与与 32 位 `int` 的加法时需保留符号，扩展成 `FFFFFFF7H`；`0000007FH` + `FFFFFFF7H` 在 32 位里得到 `00000076H`。把 `FFF7` 当成无符号 65527 再零扩展，会改变原来的负数语义；把结果仍留在 16 位，则忘了 z 是 `int`。

02 | 2009-13：自定义浮点格式先还原两个数

13. 浮点数加、减运算过程一般包括对阶、尾数运算、规格化、舍入和判溢出等步骤。设浮点数的阶码和尾数均采用补码表示，且位数分别为 5 位和 7 位（均含 2 位符号位）。若有两个数 $X = 2^7 \times 29/32$ ， $Y = 2^5 \times 5/8$ ，则用浮点加法计算 $X + Y$ 的最终结果是_____。

- | | |
|------------------|------------------|
| A. 00111 1100010 | B. 00111 0100010 |
| C. 01000 0010001 | D. 发生溢出 |

题干指定自己的阶码与尾数字段，先按题给规则解释，不套 IEEE 偏置。 $X=2^7 \times 29/32=116$ ， $Y=2^5 \times 5/8=20$ ；和是 136。把 136 写为 $2^8 \times 17/32$ ，对应阶码 `01000` 和尾数 `0010001`，选 C。关键第一笔是把两数写在同一个数值尺度上，才谈结果编码。

为何加阶码或直接加尾数字段都不行

阶码表示乘以 2 的幂，尾数表示该幂前的比例。116+20 并非指数 7+5，也不能把 29/32 与 5/8 在未对阶时直接加。另一条路是把 20 改写成 $2^7 \times 5/32$ ，于是尾数相加为 $34/32$ ，再右移一位并把阶码升为 8，得 $17/32$ 。这个“对阶→加尾数→规格化”动作后续可直接复用。

03 | 2010-14：四个等号各有不同的精度损失点

14. 假定变量 i 、 f 和 d 的数据类型分别为 `int`、`float` 和 `double` (`int` 用补码表示, `float` 和 `double` 分别用 IEEE 754 单精度和双精度浮点数格式表示), 已知 $i = 785$, $f = 1.5678e3$, $d = 1.5e100$ 。若在 32 位机器中执行下列关系表达式, 则结果为“真”的是_____。

I. $i == (\text{int})(\text{float})i$

II. $f == (\text{float})(\text{int})f$

III. $f == (\text{float})(\text{double})f$

IV. $(d+f)-d == f$

A. 仅 I 和 II

B. 仅 I 和 III

C. 仅 II 和 III

D. 仅 III 和 IV

先圈每个往返的中间类型, 不必逐个展开浮点位。整数 785 能被 `float` 精确表示, I 真; $f=1567.8\dots$ 转 `int` 会去掉小数部分, II 假; `float` 扩到 `double` 再回 `float` 保留原值, III 真; $d=1.5\times 10^{100}$ 远大于 f , 在其附近加 f 被舍去, $(d+f)-d$ 得 0, IV 假。仅 I、III, 选 B。

“double 更精确”为什么救不了第四式

`double` 的精度也是有限的。加法结果先舍入到 d 邻近的可表示值, f 比邻近值的间隔小得多时, $d+f$ 就仍是 d , 接着相减已无法找回 f 。相反 III 的输入本来只有 `float` 精度, `double` 能无损容纳这个既有值, 再转回 `float` 没有新损失。考场遇到 大数+小数-大数, 先查小数是否在第一步被吞没。

04 | 2011-13：从 -8.25 亲手填 32 位字段

13. `float` 型数据通常用 IEEE 754 单精度浮点数格式表示。若编译器将 `float` 型变量 x 分配到一个 32 位浮点寄存器 FR1 中, 且 $x = -8.25$, 则 FR1 的内容是_____。

A. C104 0000H

B. C242 0000H

C. C184 0000H

D. C1C2 0000H

第一笔把小数部分化为二进制: $8.25=1000.01_2=1.00001_2\times 2^3$ 。符号位 1; 单精度指数存 $3+127=130=10000010_2$; 尾数字段去掉隐含的首个 1, 留下 00001 后补零。合起来 1 | 10000010 | 000010000..., 即 C1040000H, 选 A。

小数位和尾数位从哪里来

$.25=1/4=0.01_2$; 1000.01_2 左移小数点三位成为 $1.00001_2\times 2^3$ 。尾数只存小数点后的位置, 不能把最前面的 1 再塞进去。先检查 C 的最高位为 1 (负号), 再检查指数和尾数, 可比在四个十六进制选项中凭外形猜快而稳。

05 | 2012-13：无符号短整数扩宽，左边补零

13. 假定编译器规定 `int` 和 `short` 型长度分别为 32 位和 16 位，执行下列 C 语言语句：

```
unsigned short x=65530;
```

```
unsigned int y=x;
```

得到 `y` 的机器数为_____。

A. 0000 7FFAH

B. 0000 FFFAH

C. FFFF 7FFAH

D. FFFF FFFAH

`unsigned short x=65530` 是 $2^{16}-6=FFFFH$ 。赋给 32 位 `unsigned int y` 时值仍为 65530，因此机器数在左边补 16 个零，0000FFFFH，选 B。和 01 的负 `short` 符号扩展对照：同样从 16 到 32 位，先看原类型是否无符号。

为什么 FFFFFFFF 看似合理却错

FFFFFFF 是 32 位的 -6 补码，或无符号值 $2^{32}-6$ 。题目原值是正的 65530，而不是有符号 `short` 的 -6。只盯着低 16 位 FFFA 会混淆“位图相同”和“解释出的值相同”；转换要保持原表达式的数值。

06 | 2012-14：最大正整数是最大有限 float 的整数值

14. `float` 类型（即 IEEE 754 单精度浮点数格式）能表示的最大正整数是_____。

A. $2^{126}-2^{103}$

B. $2^{127} 2^{104}$

C. $2^{127}-2^{103}$

D. $2^{128} 2^{104}$

单精度把指数 11111111_2 留给特殊值，所以最大有限正数用指数域 254，实际指数 127；尾数位全 1，二进制有效数是 $2-2^{-23}$ 。相乘为 $2^{128}-2^{104}$ ，选 D。这里问“能表示的最大正整数”，这个最大有限值本身就是整数，因为最低有效位处在 2^{104} 位。

为什么不把指数 255 用作更大的普通数

指数域 255 配合尾数用于无穷与 NaN，不能当普通指数 128。 $(2-2^{-23}) \times 2^{127} = 2^{128}-2^{104}$ ；减去的项来自 23 个显式尾数位。它也提醒你，“超过 2^{24} 后并非每个相邻整数都可精确表示”与“最大能表示的整数”是两种问法。

07 | 2012-15：小端处理内容，对齐处理地址

15. 某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 `int` 型和 `short` 型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

```
record.a=273;
```

若 `record` 变量的首地址为 `0xC008`，则地址 `0xC008` 中内容及 `record.c` 的地址分别为_____。

A. `0x00`、`0xC00D`

B. `0x00`、`0xC00E`

2012年计算机408统考真题 第 2 页，共 8 页

C. `0x11`、`0xC00D`

D. `0x11`、`0xC00E`

截图缺了结构体声明；原卷该处为 `int a; char b; short c;`。`record` 从 `C008H` 起，`a=273=00000111H`，小端的第一个字节是 `11H`。`a` 占 `C008—C00B`，`b` 在 `C00C`，短整数 `c` 要按 2 字节边界对齐，跳过 `C00D`，从 `C00E` 开始，故 D。

两个关键词各管一件事

“小端”管多字节数值内部的最低有效字节放在最低地址，不决定 `c` 是否填充；“边界对齐”管成员地址，短整数要从偶数地址起，不改变 `a` 的首字节。若只顺着 `4+1` 算成 `C00D`，就是漏掉填充；若首字节填 `00`，就是把大端次序套给小端。

08 | 2013-13：题面选项符号待原卷核对

13. 若某数采用 IEEE754 单精度浮点数格式表示为 `C640 0000H`，则该数的值是_____。

A. 1.5×2^{13}

B. 1.5×2^{12}

C. 0.5×2^{13}

D. 0.5×2^{12}

第一笔拆 `C6400000H` 的最高位：它是 1，故数值必为负。指数域为 140，实际指数 $140-127=13$ ；尾数对应 1.5，完整数是 $-1.5 \times 2^{13} = -12288$ 。截图中的 A 项可见为 1.5×2^{13} ，负号在截图里不可辨。请按上述可复算数值判断；若原卷选项确实均为正数，则该题选项有排印错误，不能伪称其中某个正数等于负值。

考场保底：先看最高位，立刻排除正数

IEEE 754 的最高位是符号位。这一笔不需要算阶码和尾数，便知所有纯正数选项都不可能是数值本身。阶码 140 和尾数 1.5 则进一步给绝对值 12288；若题目真实选项带负号，找到 -1.5×2^{13} 即可。后续核原卷后再定最终选项字母。

09 | 2014-14：同为负数，指数大的反而数值小

14. float 型数据常用 IEEE754 单精度浮点格式表示。假设两个 float 型变量 x 和 y 分别存放在 32 位寄存器 f_1 和 f_2 中, 若 $(f_1)=CC90\ 0000H$, $(f_2)=B0C0\ 0000H$, 则 x 和 y 之间的关系为_____。

- A. $x < y$ 且符号相同 B. $x < y$ 且符号不同
C. $x > y$ 且符号相同 D. $x > y$ 且符号不同

CC90... 与 B0C0... 的最高位都是 1，先判 同号。前者的指数域大于后者，绝对值远大于后者；负数绝对值越大，实际值越小，所以 $x < y$ 且符号相同，选 A。不必把两个大数写成十进制。

只比较十六进制大小为什么危险

IEEE 编码不能不看符号就按无符号整数排序。可拆 CC90 的指数域约为 153（实际 26），B0C0 的指数域约为 97（实际 -30）；两数绝对值差了约 56 个二进制数量级。先看符号，再比较量级，并在负号下反转大小关系。

10 | 2015-13：指定 1 的个数，造最负的 8 位补码

13. 由 3 个“1”和 5 个“0”组成的 8 位二进制补码，能表示的最小整数是_____。

- A. -126 B. -125 C. -32 D. -3

8 位补码要表示负数，最高位的 1 权重为 -128 。其余两个 1 为了让总值尽量小，就放在权重最小的末两位： $10000011_2 = -128 + 2 + 1 = -125$ ，选 B。这比在四个负数之间靠接近 -128 的感觉猜更可检验。

为什么不是 -126

-126 的补码是 10000010_2 ，只有两个 1，不满足“三个 1”。选项 -32 的位图也要按 8 位补码数 1，不能只比较数值大小。造极值的动作是先固定符号位，再把剩余的 1 安置到最小正权位置。

本批收束

01、05 是带符号与无符号扩宽对照；04、06、08、09 是浮点编码从符号、指数、尾数分别裁决；07 则把“数值的字节内容”与“成员的地址”分开。遮住答案后先做 01、05、07、09 的限时复做，若哪一步卡住，只展开相应题的机制层。后续题目继续沿用已经能复做的动作，不从零重讲。

11 | 2015-14：分清阶码越界与尾数溢出

14. 下列有关浮点数加减运算的叙述中，正确的是_____。

- I. 对阶操作不会引起阶码上溢或下溢
- II. 右规和尾数舍入都可能引起阶码上溢
- III. 左规时可能引起阶码下溢
- IV. 尾数溢出时，结果不一定溢出

A. 仅 II、III B. 仅 I、II、IV C. 仅 I、III、IV D. I、II、III、IV

先问四句各说的是阶码还是尾数。对阶只把较小指数调到已经存在的较大指数，不能凭空造出比两者更大的指数或更小的指数，I 真。右规是尾数太大时右移一位、阶码加一，可能使阶码上溢；舍入也可能让尾数进位，再右规而使阶码上溢，II 真。左规是尾数偏小时左移、阶码减一，可能下溢，III 真。尾数溢出时右规后可能仍有合法阶码，所以结果不一定溢出，IV 真。四项均真，选 D。

一条数轴把四句话放回对应步骤

对阶向已有较大指数靠齐；加尾数或舍入可能超出尾数范围，右规以指数加一补救；相消后尾数太小，左规以指数减一补救。“尾数暂时超范围”是计算中间状态，可以通过规格化挽救；“最终阶码超范围”才是阶码上溢或下溢。别把右规和对阶时为对齐而右移较小数的尾数混作同一件事。

12 | 2016-14：地址 8046 对应第几个小端字节

14. 某计算机字长为 32 位，按字节编址，采用小端（Little Endian）方式存放数据。假定有一个 double 型变量，其机器数表示为 1122 3344 5566 7788H，存放在 0000 8040H 开始的连续存储单元中，则存储单元 0000 8046H 中存放的是_____。

A. 22H B. 33H C. 77H D. 66H

从 8040H 起按字节摆 1122334455667788H。小端让最低有效字节 88 在最低地址，依次是 8040:88、8041:77、8042:66、8043:55、8044:44、8045:33、8046:22、8047:11，故 A。双精度类型此处仅说明它占 8 字节，已给机器表示，无需重新解析浮点值。

考试快做

地址差 $8046 - 8040 = 6$ ，从机器数右端起按字节数第 0、1、...、6 个即可。若从左侧数第六个，实际上用了大端方向；若按 32 位字分两组而把各组顺序颠倒，则把题设的“按字节小端”误变成了另一种约定。

13 | 2018-13：先还原补码值，再检查溢出

13. 假定带符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x、y 的值以及 x-y 的机器数分别是_____。

- A. x = -65, y = 41, x - y 的机器数溢出
- B. x = -33, y = 65, x - y 的机器数为 FFFF FF9DH
- C. x = -33, y = 65, x - y 的机器数为 FFFF FF9EH
- D. x = -65, y = 41, x - y 的机器数为 FFFF FF96H

FFFFFDFH 作为 32 位补码，等于 -33；00000041H=65。先按数学值算 $x-y=-98$ ，显然处在 32 位有符号范围内，补码是 FFFFFFF9EH，选 C。看见很多 F 时别把它当成巨大正数；题干已经指定有符号补码。

无需先做整串十六进制借位

0xDF 的低字节相对 0x100 少 0x21=33；差值 -98 的 8 位补码是 $256-98=158=9EH$ ，符号扩展到 32 位便是 FFFFFFF9EH。同号相减或者这里一个负数减正数，确实可能下溢，但 -98 离 -2^{31} 很远，不能仅凭符号组合就宣布溢出。

14 | 2018-14：最小规格化值与最小非零值分开

14. IEEE 754 单精度浮点格式表示的数中，最小的规格化正数是_____。

- A. 1.0×2^{-126}
- B. 1.0×2^{-127}
- C. 1.0×2^{-128}
- D. 1.0×2^{-149}

“规格化正数”要求指数域至少为 1，隐含的有效数首位是 1。指数域 1 对应实际指数 $1-127=-126$ ，尾数全零，故最小规格化正数 1.0×2^{-126} ，选 A。 2^{-149} 更小，却属于非规格化数，不满足题目限定。

为什么指数域零时不再有隐含 1

指数域全零用于零与非规格化数。它们用 0.尾数字段 $\times 2^{-126}$ ，可表示比最小规格化正数更接近 0 的数；尾数最低一位为 1 时就是 2^{-149} 。题干若删去“规格化”，答案判定会改变；这里这四个字就是裁决选项 D 的证据。

15 | 2018-15：指令前三字节不动，只替换立即数

15. 某 32 位计算机按字节编址，采用小端（Little Endian）方式。若语句“int i = 0;”对应指令的机器代码为“C7 45 FC 00 00 00 00”，则语句“int i = -64;”对应指令的机器代码是_____。

- A. C7 45 FC C0 FF FF FF
- B. C7 45 FC 0C FF FF FF
- C. C7 45 FC FF FF FF C0
- D. C7 45 FC FF FF FF 0C

两句赋值只有常数从 0 变为 -64。题目给 int i=0 对应 C7 45 FC 00 00 00 00，因此先保留前 3 个指令字节，只改后 4 个立即数字节。 -64 的 32 位补码 FFFFFFFC0H，小端依次存 C0 FF FF FF，整个机器码 C7 45 FC C0 FF FF FF，选 A。

为什么不能把整条指令也反过来

小端规定的是一个多字节数据的字节顺序，不是把不同字段构成的机器指令整体倒序。C7 45 FC 是原题已给出的操作码和寻址部分；C0 FF FF FF 是这个立即数自身的低字节到高字节。选 C 把立即数按大端放，选 D 则连补码的低字节都写错。

16 | 2019-13：同样的 FFFF，换成 signed short 就读作 -1

13. 考虑以下 C 语言代码：

```
unsigned short usi = 65535;
short si = usi;
```

执行上述程序段后，si 的值是_____。

- A. -1 B. -32767 C. -32768 D. -65535

unsigned short usi=65535 的 16 位图是 FFFFH。赋给同宽 short si，按这道题采用的补码机器解释，相同 16 位全 1 表示 -1，选 A。承接 05：那题目标类型是更宽的 unsigned int，数值 65530 可以无损保留；这里目标 short 容不下正的 65535。

语言规范和统考机器题的边界

C 语言标准对超出 signed short 范围的无符号值转换，允许实现定义的结果或实现定义的信号；题目预设常见补码机器行为，四个选项据此取 -1。实际写可移植 C 程序时，不能把这种转换当成跨平台保证。这里教学目标是识别目标位宽内的机器数解释，且不误以为 -65535 能塞进 16 位 short。

17 | 2019-15：基址求首地址，大端再找末字节

15. 某计算机采用大端方式，按字节编址。某指令中操作数的机器数为 1234 FF00H，该操作数采用基址寻址方式，形式地址（用补码表示）为 FF12H，基址寄存器的内容为 F000 0000H，则该操作数的 LSB（最低有效字节）所在的地址是_____。

- A. F000 FF12H B. F000 FF15H C. EFFF FF12H D. EFFF FF15H

操作数 1234FF00H 的最低有效字节是 00H，其余三字节依次为 FF, 34, 12。题设大端，首地址放 12，LSB 在首地址加 3。基址寄存器 F0000000H 与题目给出的 16 位形式地址 FF12H 组成 F000FF12H，故 LSB 地址 F000FF15H，选 B。

地址形成的隐含宽度要和选项一起核验

四个选项把前半部限定为 F000 或 FFFF，后半部为 FF12 或 FF15。此题按 16 位无符号位移量 FF12H 加到基址，得到 F000FF12H。若把 FF12H 当成 16 位有符号 -238 再符号扩展，32 位和会成为 EFFFFFF12H，选项中没有；题干写“形式地址（用补码表示）”容易诱导这种读法，需注意题库所用地址形成约定并未说清符号扩展。确定的另一半是大端的 LSB 必在首地址+3。

18 | 2020-13：先看 x 的类型是 int 还是 float

13. 已知带符号整数用补码表示，float 型数据用 IEEE 754 标准表示，假定变量 x 的类型只可能是 int 或 float，当 x 的机器数为 C800 0000H 时，x 的值可能是（ ）。

- A. -7×2^{27} B. -2^{16} C. 2^{17} D. 25×2^{27}

机器数 C8000000H 若按有符号 32 位 int 解释，值为 $0xC8000000 - 2^{32} = -0x38000000 = -7 \times 2^{27}$ ，与 A 相符。若按 float，符号位 1、指数域 144（实际 17）、尾数 0，值为 -2^{17} ，不对应其余选项。题干说 x 只可能属于两种类型，意味着必须对每种解释各算一次，而不能凭 C8 开头猜一个类型。

为什么负整数的绝对值不是 C8 后面补零

补码的最高位权重为负，C8H=11001000₂ 在最高字节对应 $-128+64+8=-56$ ，后面三个零字节相当于乘 2^{24} ，即 $-56 \times 2^{24} = -7 \times 2^{27}$ 。浮点却用同一最高位表示符号、其后八位表示偏置阶码；类型不同，位的分工不同。这一题正是本节点的总判断。

19 | 2020-14：结构体偏移和小端字节偏移分别加

14. 在按字节编址，采用小端方式的 32 位计算机中，按边界对齐方式为以下 C 语言结构型变量 a 分配存储空间：

```
struct record{
    short  x1;
    int    x2;
} a;
```

若 a 的首地址为 2020 FE00H，a 的成员变量 x2 的机器数为 1234 0000H，则其中 34H 所在存储单元的地址是（ ）。

- A. 2020 FE03H B. 2020 FE04H C. 2020 FE05H D. 2020 FE06H

short x1 从 2020FE00H 占两个字节；为了让 int x2 按 4 字节边界对齐，跳过两个填充字节，x2 首地址在 2020FE04H。它的机器数 12340000H 按小端从低到高写成 00 00 34 12；34H 在 x2 偏移 2，即 2020FE06H，选 D。

为何不能看到 short 两字节就立即加 2

FE02H 是 x1 后的地址，却还不是 4 字节边界；x2 从 FE04H 起。然后要在这个成员的四个字节里找 34H，而不是在结构体首址按机器数 12340000H 直接找。先求成员起点，再求目标字节偏移，两个加法服务不同问题。

20 | 2021-14：十进制小数何时能以二进制有限表示

14. 下列数值中，不能用 IEEE 754 浮点格式精确表示的是（ ）。

A. 1.2

B. 1.25

C. 2.0

D. 2.5

先把选项的小数写成最简分数。1.2=6/5 的分母含 5，不可能写成有限个二进制位；1.25=5/4、2.0=2、2.5=5/2 都能有限表示，而且所需位数很少，故不能精确表示的是 A。这一题不必展开 IEEE 的 32 个字段。

分母判据的边界

有限二进制小数总能写成 整数/ 2^k ，约分后分母只能含因子 2。反过来，最简分数分母仅有 2 时可有限展开；仍要检查指数与有效位数是否落在所问格式的范围，这四个普通小数满足。1.2 即使存入 double 也只是近似值，更多位数不把循环二进制展开变成有限展开。

11—20 的加速收束

编码题先拆类型、符号、宽度；地址题先算对象起点再定位第几个字节；浮点题看题干限制的是“规格化”“精确表示”还是运算后的溢出。考场通常只需抓这一句限制，机制层留给遮答案复做时确实卡住的地方。

21 | 2022-13：32 位补码的两端不对称

13. 32 位补码所能表示的整数范围是（ ）。

A. $-2^{32} \sim 2^{31} - 1$

B. $-2^{31} \sim 2^{31} - 1$

C. $-2^{32} \sim 2^{32} - 1$

D. $-2^{31} \sim 2^{32} - 1$

先把最高位的负权 -2^{31} 单独放在左边。其余 31 位全零时取最小值 -2^{31} ；其余 31 位全一且最高位为零时取最大值 $2^{31} - 1$ 。范围 $[-2^{31}, 2^{31} - 1]$ ，选 B。负端比正端多一个，因为零占了一个非负编码。

不要把 32 位都当正权位

100...0 的补码是 -2^{31} ，不是 -2^{32} ；011...1 有 31 个正权位，和为 $2^{31} - 1$ 。111...1 反而是 -1。这里问“所有能表示的整数”，无需讨论负零：补码只有一个零，因而区间呈现这种一位差。

22 | 2022-14：小数化二进制后再编码浮点

14. -0.4375 的 IEEE 754 单精度浮点数表示为（ ）。

A. BEE0 0000H

B. BF60 0000H

C. BF70 0000H

D. C0E0 0000H

0.4375=7/16=0.0111₂=1.11₂× 2^{-2} 。负号位 1，偏置指数 $-2+127=125=01111101_2$ ，尾数 1100...0。拼出 BE600000H，选 B。它承接 04 的编码动作，区别在于小于 1 的数先向右移动小数点才得到规格化形式。

拆回去验证，避免十六进制抄错

BE60... 最高位为 1；后八位还原指数域 125；显式尾数头两位是 11，有效数为 $1.11_2=1.75$ ，乘 2^{-2} 为 0.4375。BEE0... 的尾数不同，BF60... 的指数不同，不能只看最高十六进制字节。

23 | 2023-13：负数补码用 2^{16} 减绝对值

13. 若 short 型变量 $x = -8190$ ，则 x 的机器数是()。

A. E002H

B. E001H

C. 9FFFH

D. 9FFE H

题目问 short 的机器数，按 16 位算： $2^{16}-8190=65536-8190=57346=E002H$ ，选 A。便捷草稿是 $8190=0x1FFE$ ，逐位取反加一得到 E002。别用 32 位补码 $FFFFE002$ 去匹配 16 位选项。

两种检验同一结果

$0xE002$ 作为有符号 16 位，数值为 $57346-65536=-8190$ ；或者低 16 位 $0x1FFE+0xE002=0x10000$ ，截断进位得零。E001 少一、9FFE 对应另一种错误的负权，检验比凭印象认首位快。

24 | 2023-14：指数全零要切到非规格化规则

14. 已知 float 型变量用 IEEE 754 单精度浮点数格式表示。若 float 型变量 x 的机器数为 8020 0000H，则 x 的值是()。

A. -2^{-128}

B. -1.01×2^{-127}

C. -1.01×2^{-126}

D. 非数 (NaN)

80200000H 的符号位为 1，指数域全零，尾数非零，所以是非规格化负数，不是 NaN。此时有效数写作 $0.01_2=1/4$ ，实际指数固定为 -126 ，故数值 $-(1/4) \times 2^{-126}=-2^{-128}$ ，选 A。

为什么 B、C 的 1.01 前缀不适用

规格化数才有隐藏的首位 1；指数域全零时隐藏位改为 0。尾数的首个 1 出现在小数点后第二位，因此是 $1/4$ 。指数全 1 且尾数非零才表示 NaN，不能把“特殊指数域”笼统称为非数。

12. C 语言代码如下。

```
int i=32777;  
short si=i;  
int j=si;
```

执行上述代码段后，j 的值是（ ）。

A. -32 777

B. -32 759

C. 32 759

D. 32 777

32777 比 16 位 signed short 最大值 32767 大 10。按题目采用的 16 位补码转换，短整数的同一低 16 位表示 $32777 - 65536 = -32759$ ；再赋给 int j 是符号扩展，数值仍 -32759 ，选 B。承接 16，明确第一次窄化已经改变了可表示数值，第二次扩宽不会把丢失的范围找回来。

代码语义的边界

在标准 C 中，超出 short 范围的 int 转 short，其结果由实现定义或发出实现定义的信号；统考默认常见补码保留低 16 位。草稿先判断是否越界，若越界且题目以机器补码命题，再减 2^{16} 。切勿写成 -32777 ：那不是由低位截取产生的结果。

26 | 2024-14：大范围整数参数先问是否每个值都精确

14. 某科学实验中，需要使用大量的整型参数，为了保证表数精度的基础上提高运算速度，需要选择合理的数据表示方法。若整型参数 α 、 β 的取值范围分别为 $-2^{20} \sim 2^{20}$ 、 $-2^{40} \sim 2^{40}$ ，则下列选项中， α 、 β 最适宜采用的数据表示方法分别是（ ）。

- A. 32 位整数、32 位整数
- B. 单精度浮点数、单精度浮点数
- C. 32 位整数、双精度浮点数
- D. 单精度浮点数、双精度浮点数

α 取到 $\pm 2^{20}$ ，32 位有符号整数完整覆盖； β 取到 $\pm 2^{40}$ ，32 位整数放不下，而单精度 float 只有 24 位有效二进制精度，无法保证这个区间每个整数精确。double 有 53 位有效二进制精度，能精确覆盖 $\pm 2^{40}$ 内的整数。故 α 用 32 位整数、 β 用双精度浮点数，选 C。

“能表示 2^{40} ”不等于“能精确表示附近所有整数”

单精度 float 的指数足以到 2^{40} ，但那附近相邻可表示数间隔远超过 1，会跳过大量整数。题干强调大量整型参数、保证精度并兼顾速度： α 用 int 避免浮点开销； β 在给定选项里用 double 覆盖所需整数精度。若允许 64 位整数，会是另一种自然选择，但四项没有这一组合。

27 | 2025-12：负 short 赋给 unsigned int 先扩宽后取模

12. 在 32 位计算机上执行下列 C 语言代码，则 ui 的真值为

```
short si = -32767;
```

```
unsigned int ui = si;
```

- A. $2^{13}-1$ B. $2^{15}+1$ C. $2^{32}-2^{15}-1$ D. $2^{32}-2^{15}+1$

$si = -32767 = -(2^{15}-1)$ 。赋给 32 位 unsigned int 时，最终值按 2^{32} 取模： $2^{32}-32767 = 2^{32}-2^{15}+1$ ，选 D。先记住负数并没有突然变为 32769；它变成很大的无符号 32 位值。

沿类型转换链看位图

short 的 16 位补码为 8001H；整数提升先保留符号扩为 32 位 FFFF8001H，按 unsigned int 解读后等于 $2^{32}-32767$ 。若错误地零扩展 8001H 得 32769，就相当于先把 si 当 unsigned short，和题干的 signed short 不同。05 恰好提供这种有符号与无符号扩宽的对照。

28 | 2025-13 : 47300000 拆出 1.375 与 2^{15}

13. 已知 float 型变量用 IEEE754 单精度浮点数格式表示。若 float 型变量 x 的机器数为 4730 0000H；则 x 的值为

- A. 0.375×2^{14} B. 1.375×2^{14} C. 0.375×2^{15} D. 1.375×2^{15}

最高位 0，数为正；4730... 的指数域为 142，真实指数 $142-127=15$ ；尾数字段对应 $0.011_2=0.375$ ，隐藏位补成 $1.011_2=1.375$ 。所以 1.375×2^{15} ，选 D。A、C 都忘了隐藏的 1，B 把指数少算一位。

用数值回代核对字段

0x47300000 的后 23 位是 0x300000，它占尾数字段的 3/8，等于 0.375；指数域可按 $(0x47300000 \gg 23) \& 0xFF$ 得 142。二进制等价 $1.011_2 \times 2^{15}$ ，不需要把它完全展开成十进制整数。

29 | 2025-15 : 结构体数组再多一层“步长”

15. 某 32 计算机按字节编制，采用小端方式存放数据，编译器按边界对齐方式为下列 C 语言结构型数组变量 employee 分配储存空间。数组 employee 的起始地址为 0000A0B0H，employee[1].id 的机器数为 12345678H，则 56H 的地址是

struct record

```
{
    int id;
    char name[10];
    int salary;
} employee[200];
```

- A. 0000 A0C3H B. 0000 A0C4H C. 00000 A0C5H D. 0000 A0C6H

先算每个记录的大小：id 4 字节，name[10] 10 字节，接着填 2 字节使 salary 从偏移 16 开始，再占 4 字节，每条记录 20 字节（14H）。数组首址 A0B0H，employee[1].id 在 A0C4H。12345678H 小端字节顺序是 78 56 34 12，56H 位于 id 的第 1 个偏移，地址 0000A0C5H，选 C。

把 07、19 的地址动作叠起来

07 只求同一个记录内的成员地址；19 再求成员内部某字节。本题在此前多了数组下标：数组首址 + 下标 × 结构体步长 + 成员偏移 + 字节偏移。这里只有 id 成员偏移 0，但结构体尾部填充影响数组步长。若直接用 $4+10+4=18$ 乘下标，会让下一记录错位；若看到机器数中的 56 从左数，会忘记小端。

0812 收束 | 把长推理压成三次询问

遮住答案拿到一题时，先问“位图按什么类型和位宽解释？”，再问“题目要数值、精度，还是某个地址的字节？”，最后只执行相应的小动作：补码取模、浮点符号阶码尾数、成员对齐加字节偏移。三步都能独立完成，才把本节点记为会解；能在一两分钟内稳定复做不同年份变式，才记为熟练。08 的卷面选项与编码矛盾、17 的地址位移宽度含混，复做时应保留这两个题面异常，不把猜中的选项当成掌握证据。